Online Appendix to: Community Discovery in Dynamic Networks: A Survey

GIULIO ROSSETTI, ISTI-CNR
REMY CAZABET, University of Lyon & Sarbonne University

In this section, we propose a brief description for each DCD method that we have identified in the literature, framing them in the proposed taxonomy. When possible, we contacted the authors to validate our description. Inside categories, methods are ordered chronologically.

A INSTANT-OPTIMAL COMMUNITY DISCOVERY

Here are collected and classified all of the works introducing approaches falling in the *Instant-optimal* class. For details on the class definition and peculiarities, refer to Section 4.1.

A.1 Iterative Similarity-Based Approaches

The algorithm in Hopcroft et al. (2004) can be seen as the ancestor of all approaches within this category. It first finds "natural communities" in each snapshot by running the same algorithm for community detection several times and maintaining only those communities that appear in most runs. In a second phase, similar communities of successive snapshots are matched together using a match function defined as follows:

$$match(C, C') = Min\left(\frac{|C \cap C'|}{|C|}, \frac{|C \cap C'}{|C'|}\right),$$

with C and C' the clusters to compare treated as sets.

The algorithm proposed in Palla et al. (2007) is based on clique percolation (CPM (Palla et al. 2005)). CPM is applied on each snapshot to find communities. The matching between these communities is done by running CPM on the joint graph of snapshots at t and $t + 1$. Due to the formal definition of communities provided by CPM (communities are composed of cliques), communities of the joint graph can only grow compared to those of both snapshots. The authors also discuss how the knowledge of the temporal commitment of members to a given community can be used for estimating the community's lifetime. They observed that although the stability of small communities is ensured by the high homogeneity of the nodes involved, for large ones the key to a long life is continuous change.

In Bourqui et al. (2009), an approach based on dynamic graph discretization and graph clustering is proposed. The framework allows detection of major structural changes over time, identifies events by analyzing temporal dimensions, and reveals hierarchies in social networks. It consists of four major steps:

(1) The social network is decomposed into a set of snapshot graphs.
(2) Clusters are extracted from each static graph separately using an overlapping CD algorithm, to produce overlapping structures. This step allows identification of communities in the network but also its pivots (vertices shared by several clusters) while being insensitive to minor changes in the network.
(3) Major structural changes in the network are detected by comparing the clustering obtained on every pair of successive static graphs using a similarity measure (*cluster*

© 2018 ACM 0360-0300/2018/02-ART35 $15.00
https://doi.org/10.1145/3172867

representativeness, e.g., the geometrical mean of the normalized ratio of common elements between two clusters). Thus, the temporal changes in the input network are decomposed into periods of high activity and consensus communities during stable periods.

(4) An influence hierarchy in the consensus communities is identified using a minimum spanning tree.

In Asur et al. (2009), the authors approach the problem of identifying events in communities: MCL (a modularity-based algorithm) is applied to extract communities in each timestep. A set of events (Continue, kMerge, kSplit, Form, Dissolve, Join, Leave) are defined. For instance, kMerge is defined as follows: $kMerge(C_i^k, C_i^l, k) = 1$ if and only if $\exists C_{i+1}^j$ such that

$$\frac{|(V_i^k \cup V_i^l) \cap V_{i+1}^j|}{Max(|V_i^k \cup V_i^l|, |V_{i+1}^j|)} > k\%$$

and $|V_i^k \cap V_{i+1}^j| > \frac{|C_i^k|}{2}$ and $|V_i^l \cap V_{i+1}^j| > \frac{|C_i^l|}{2}$, with i representing timesteps.

In Rosvall and Bergstrom (2010), the authors propose a method to map changes in large networks. To do so, they focus on finding stable communities for each snapshot by using a bootstrap resampling procedure accompanied by significance clustering. More precisely, for each snapshot, 1,000 variations of the original network are generated, in which the weight of each edge is taken randomly from a Poisson distribution with the original weight of the edge as the mean. All of these alternative networks are clustered using a static algorithm. To find significant communities, the authors propose to search for modules—largest sets of nodes that are clustered together in more than 95% of networks. These small node sets are grouped into larger clusters if their nodes appear together in more than 5% of network instances. Clusters corresponding to different timesteps are linked if the number of common nodes is higher than a user-defined threshold.

In Bóta et al. (2011), the authors propose an extension of the basic community events described in Palla et al. (2007) and an approach capable of handling communities identified by a nonmonotonic community detection algorithm (N^{++} (Bóta et al. 2010)). The proposed method works on snapshots and uses, for each pair of adjacent network observation, *union* (and *intersection*) graphs to perform community matching. The final goal of this approach is to build an extended community life cycle. In addition to the classical events defined in Palla et al. (2007), five metaevents are introduced to capture the combinations of standard events: grow-merge, contraction-merge, grow-split, contraction-split, and obscure case (which envelop more complex, nonbinary transformations).

The method proposed in Takaffoli et al. (2011) is designed to track the evolution of communities. First, a static community detection is run on each snapshot, using an existing algorithm. In a second step, communities matching and operations are done according to a set of rules that can be summarized as follows:

—Two communities are matched if they have at least a fraction k of the nodes of the largest one in common.
—A new community is born if it has no match in the previous snapshot.
—A community dies if it has no match in the following snapshot.
—A community splits if several other communities have more than a fraction k of its nodes.
—A merge occurs if a community contains at least a fraction k of several other communities.

The method proposed in Greene et al. (2010) is designed to track the evolution of communities in time evolving graphs following a two-step approach: (i) community detection is executed on each snapshot with a chosen static algorithm and then (ii) the Jaccard coefficient is used to match

communities across adjacent snapshots. Communities are matched if their similarity is above a user-defined threshold, allowing many-to-many mappings.

The GED method, proposed in Bródka et al. (2013), is designed to identify what happened to each node group in successive time frames: it uses not only the size and equivalence of groups' members but also takes into account their position (via node ranking within communities, social position, etc.) and importance within the group. This approach is sustained by a new measure called *inclusion*, which respects both the *quantity* (the number of members) and *quality* (the importance of members) of the group. The proposed framework is parametric in the static CD algorithm to be applied on each network snapshot to extract node groups (communities).

The method proposed in Dhouioui and Akaichi (2014) uses the OCD algorithm (Dhouioui and Akaichi 2013) to find overlapping communities in each snapshot. Then relations are found between similar communities in different snapshots. A Data Warehouse layer memorizes the evolution of the network and its communities.

In İlhan and Öğüdücü (2015), an evolving network is represented by a series of static snapshots. Communities are determined using the Louvain algorithm (Blondel et al. 2008), then matched using a custom similarity metric. The evolution of communities is labeled with the events they experienced (survive, grow, shrink, merge, split, and dissolve). Moreover, the authors propose to forecast future evolution of communities. To do so, a broad range of structural and temporal features are extracted covering many properties of both the internal link structure and the external interaction of each community with the rest of the network. A temporal window length is defined to generate time series for each feature; thus. for each community, a set of time series is built. Then, the autoregressive integrated moving average (ARIMA) model (Newbold 1983) is applied to estimate the next values of the features. The communities with the forecasted feature values were then used as test set for several classification algorithms trained using the rest of the window length snapshots as the training set.

A.2 Iterative Core Nodes–Based Approaches

The method described in Wang et al. (2008) proposes to use the concept of core nodes to track the evolution of communities. It is agnostic with respect to the CD algorithm used to detect communities on each snapshot. For each snapshot, a voting strategy is used to select a few core nodes. Finally, the tracking of communities can be done by tracking the core nodes.

In Chen et al. (2010), the authors adopt a conservative definition of communities–that is, they define them as the maximal cliques of the graph. They consequently introduce *graph representatives* and *community representatives* to limit the search space and avoid redundant communities. They are defined as follows:

— *Graph representatives*: Representatives of graph G_t are the nodes that also appear in G_{t1}, G_{t+1}, or both. Nodes that only appear in one graph are called *graph-dependent nodes*. If a community only contains graph-dependent vertices, then it can be considered as a "graph-dependent" community, not a dynamic one
— *Community representatives*: A community representative of community C_i^t is a node in C_i^t that has the minimum number of appearances in other communities of the same graph G_t.

The proposed approach first finds graph representatives and enumerates the communities that are seeded by the graph representatives to avoid considering redundant communities. Then, in every generated community, it selects only one node as a community representative and uses community representatives to establish the predecessor/successor relationship between communities of different timesteps. Once all predecessors and successors have been found, it finally applies decision rules to determine community dynamics.

A.3 Multistep Matching

In Falkowski et al. (2006) and Falkowski and Spiliopoulou (2007), a three-step approach is applied to detect subgroups in social networks:

(1) In the first step, communities are found in each snapshot using a static CD algorithm.
(2) In the second step, communities in different snapshots are linked based on their similarity, using the overlap measure: $overlap(x, y) = \frac{|x \bigcap y|}{min(|x|, |y|)}$. This creates a *community survival graph*.
(3) In the third step, a community detection algorithm is run on the community survival graph, thus finding communities relevant across several snapshots.

The approach proposed in Goldberg et al. (2011) identifies evolutive chains of communities. Given a time-evolving graph, community detection on each snapshot is executed using a chosen static algorithm (including overlapping ones). Any set intersection-based measure can be used to match communities between snapshots, creating a link between them. The authors propose a strategy to find the best chains of evolution for each community: they define chain strength as the strength of its weakest link. As a result, all maximal valid chains are constructed for the identified communities. A valid chain is considered maximal if it is not a proper subchain of some other valid chain.

In Morini et al. (2017), the authors start by computing aggregated network snapshots from timestamped observations (SN or TN) using sliding windows. Communities are detected using a static CD algorithm on each of these windows independently. Then the similarity between a pair of communities is computed between communities at t and communities at $t-2$, $t-1$, $t+1$, $t+2$. This information is used to smooth out the evolution of communities: if a community c_{t-n} in a previous snapshot has a high similarity with c_{t+n} in a later one but is matched with lower similarity to two communities $c1_t$ and $c2_t$ at t, then these two communities are merged and the resulting community is identified to c_{t-n} and c_{t+n}. The same mechanism is used to smooth out artificial merges.

B TEMPORAL TRADE-OFF COMMUNITIES DISCOVERY

Here we have collected and classified all works introducing approaches falling in the Temporal Trade-off class. For details on the class definition and peculiarities, refer to Section 4.2.

B.1 Partition Update by Global Optimization

In Miller and Eliassi-Rad (2009), the authors adapt a dynamic extension of latent Dirichlet allocation to the extraction of dynamic communities. They design cDTM-G, the "continuous-time dynamic topic model for graphs." In this work, observations of the time-evolving graph (snapshots) correspond to *corpora* at different points in time: each source node at each timestep corresponds to a document at the same timestep, and the links among the nodes connect document's words. Exploiting such modeling strategy, dynamic groups/communities capture time-evolving topics. The authors sequentially run LDA-G (Henderson and Eliassi-Rad 2009) on each timestep initializing the topics for the current timestep with the ones learned in the previous timestep.

The method proposed in Aynaud and Guillaume (2010) is based on the Louvain algorithm. In the original, static version, the Louvain algorithm is initialized with each node in its community. A greedy, multistep heuristic is used to move nodes to optimize the partition's modularity. To limit instability, the authors propose to initialize the Louvain algorithm at t by communities found at $t-1$. A parameter $x \in [0-1]$ can be tuned to specify a random fraction of nodes that will start in a singleton community, to avoid staying in a local minimum (community drift).

The algorithm proposed in Görke et al. (2010) efficiently maintains a modularity-based clustering of a graph for which dynamic changes arrive as a stream. The authors design dynamic generalizations of two modularity maximization algorithms Clauset et al. (2004) and Blondel et al. (2008). The problem is cast as an integer linear programming (ILP). Intuitively, the process at t is initialized by communities found at $t-1$, adding a backtracking strategy to extend the search space.

In Bansal et al. (2011), the authors introduce a dynamic community detection algorithm for real-time online changes, which involves the addition or deletion of edges in the network. The algorithm, based on the greedy agglomerative technique of the CNM algorithm (Clauset et al. 2004), follows a hierarchical clustering approach, where two communities are merged at each step to optimize the increase in the modularity of the network. The proposed improvement for the dynamic case consists of starting from the dendrogram of the previous step cut just before the first merge of a node involved in the network modifications. It assumes that the small change in network size and density between snapshots does not dramatically impact modularity for nonimpacted nodes, and therefore the beginning of the dendrogram is not impacted.

The method proposed in Shang et al. (2014) searches for evolving communities of maximal modularity. The Louvain algorithm (Blondel et al. 2008) is used to find communities in the first snapshot. Custom rules are then applied to each modification of the network to update the communities while keeping the modularity as high as possible. The procedure designed to update the communities depends on the type of modification of the network, which can concern an inner community edge, an across community edge, a half-new edge (if one of the involved nodes is new), or a new edge (if both extremities are new nodes).

The authors of Alvari et al. (2014) introduce a game-theoretic approach for community detection in dynamic social networks called D-GT: in this context, each node is treated as a rational agent who periodically chooses from a set of predefined actions to maximize its utility function. The community structure of a snapshot emerges after the game reaches Nash equilibrium: the partitions and agent information are then transferred to the next snapshot. D-GT attempts to simulate the decision-making process of the individuals creating communities rather than focusing on statistical correlations between labels of neighboring nodes.

B.2 Informed CD by Multiobjective Optimization

In Zhou et al. (2007), the authors seek communities in bipartite graphs. Temporal communities are discovered by threading the partitioning of graphs in different periods, using a constrained partitioning algorithm. Communities for a given snapshot are discovered by optimizing the *normalized cut*. The discovery of community structure at time t seeks to minimize the (potentially weighted) sum of distances between the current and n previous community membership, with n being a parameter of the algorithm.

The approach proposed in Tang et al. (2008) allows finding the evolution of communities in multimode networks. To find communities at time t, the authors propose using a spectral-based algorithm that minimize a cost function defined as the sum of two parts: F_1 and Ω. F_1 corresponds to the reconstruction error of a model similar to block modeling but adapted to multimode networks. Ω is a regulation term that states the difference between the current clustering and the previous one. Weights allow balancing the relative importance of these two factors.

In Yang et al. (2009, 2011), the authors propose using a dynamic SBM. As in a typical static SBMs, nodes belong to clusters, and an interaction probability β_{ql} is assigned to each pair of clusters (q, l). The dynamic aspect is handled by a transition matrix that determines the probability for nodes belonging to a cluster to move to each other cluster at each step. The optimal parameters of this

model are searched for using a custom EM algorithm. The β_{ql} are constant for the same community for all timesteps.

Two versions of the framework are discussed:

— *online learning*, which updates the probabilistic model iteratively (in this case, the method is a Temporal Trade-Off CD, similar to FacetNet (Lin et al. 2009)), and
— *offline learning*, which learns the probabilistic model with network data obtained at all timesteps (in this case, the method is a Cross-Time CD).

Several variations with a similar approach have been proposed by different authors, most notably in Ishguro et al. (2010), Herlau et al. (2013), and Xu and Hero (2014). The methods discussed in Folino and Pizzuti (2010, 2014) adopt a genetic algorithm to optimize a multiobjective quality function. One objective is to maximize the quality of communities in the current snapshot (the modularity is used in the article, but other metrics such as conductance are proposed). The other objective concerns the maximization of the NMI between the communities of the previous step and of the current one to ensure a smooth evolution.

In FacetNet, introduced in Lin et al. (2008, 2009), the authors propose finding communities at t. To minimize the reconstruction error, a cost error defined as $cost = \alpha C_1 + (1 - \alpha)C_2$ is defined, with C_1 corresponding to the reconstruction error of a mixture model at t and C_2 corresponding to the KL divergence between clustering at t and clustering at $t - 1$. The authors then propose a custom algorithm to optimize both parts of the cost function. This method is similar to Yang et al. (2009).

The method described in Sun et al. (2010, 2014) proposes finding multityped communities in dynamic heterogeneous networks. A Dirichlet process mixture model–based generative model is used to model the community generations. A Gibbs sampling–based inference algorithm is provided to infer the model. At each timestep, the communities found a trade-off between the best solution at t and a smooth evolution compared to the solution at $t - 1$.

In Gong et al. (2012), the detection of community structure with temporal smoothness is formulated as a multiobjective optimization problem. As in Folino and Pizzuti (2010, 2014), the maximization is performed on modularity and NMI.

In Kawadia and Sreenivasan (2012), a new measure of partition distance is introduced. *Estrangement* captures the inertia of internode relationships that, when incorporated into the measurement of partition quality, facilitates the detection of temporal communities. Upon such a measure is built the *estrangement confinement method*, which postulates that neighboring nodes in a community prefer to continue to share community affiliation as the network evolves. The authors show that temporal communities can be found by estrangement constrained modularity maximization, a problem they solve using Lagrange duality. Estrangement can be decomposed into local single-node terms, thus enabling an efficient solution for the Lagrange dual problem through agglomerative greedy search methods.

In Crane and Dempsey (2015), a hidden Markov model is introduced for inferring community structures that vary over time according to the cut-and-paste dynamics from Crane (2014). The model takes advantage of temporal smoothness to reduce short-term irregularities in community extraction. The partition obtained is without overlap, and the network population is fixed in advance (no node appearance/vanishing).

In Görke et al. (2013), an algorithm is proposed to efficiently maintain a modularity-based clustering of a graph for which dynamic changes arrive as a stream, similar to Görke et al. (2012). Differently from such work, here the authors introduce an explicit trade-off between modularity maximization and similarity to the previous solution, measured by the Rand index. As in other methods, a parameter α allows tuning the importance of both components of this quality function.

B.3 Partition Update by a Set of Rules

DENGRAPH (Falkowski et al. 2008) is an adaptation of the data clustering technique DBSCAN (Ester et al. 1996) for graphs. A function of proximity ranging in [0,1] is defined to represent the distance between each pair of vertices. A vertex v is said to be density reachable from a core vertex c if and only if the proximity between v and c is above a parameter ω. A core vertex is defined as a vertex that has more than η density-reachable vertices. Communities are defined by the union of core vertices that are density reachable from each other. A simple procedure is defined to update the communities following each addition or removal of an edge.

Adaptative FOCS (AFOCS), described in Nguyen et al. (2011a), uses the FOCS algorithm to find initial communities. Communities found by FOCS are dense subgraphs that can have strong overlaps. Starting from such topologies, a local procedure is applied to update the communities at each modification of the network (e.g., addition or removal of a node or an edge). The strategy, proposed to cope with local network modifications, respects the properties of communities as they are defined by the FOCS algorithm.

In Cazabet et al. (2010), iLCD is introduced, an incremental approach able to identify and track dynamic communities of high cohesion. Two measures are defined to qualify the cohesion of communities:

— *EMSN*: the estimation of the mean number of second neighbors in the community; and
— *EMRSN*: the estimation of the mean number of robust second neighbors (second neighbors that can be accessed by at least two different paths).

Given a new edge, the affected nodes can join an existing community if an edge appeared with a node in this community. The condition is that the number of its second neighbors in the community is greater than the value of EMSN of the community, and respectively for EMRSN. iLCD has two parameters: k and t. The former regulates the rising of new communities (new communities are created if a new clique of size k appears outside of existing ones), whereas the latter regulates the merge of existing ones (communities can be merged if their similarity becomes greater than a threshold t).

In Nguyen et al. (2011b), the quick community adaptation (QCA) algorithm is introduced. QCA is a modularity-based method for identifying and tracking community structure of dynamic online social networks. The described approach efficiently updates network communities, through a series of changes, by only using the structures identified from previous network snapshots. Moreover, it traces the evolution of community structure over time. QCA first requires an initial community structure, which acts as a basic structure, to process network updates (node/edge addition/removal).

In Cazabet and Amblard (2011), the authors propose using a multiagent system to handle the evolution of communities. The network is considered as an environment, and communities are agents in this environment. When what they perceive of their local environment changes (edge/node addition/removal), community agents make decisions to add/lose nodes or merge with another community based on a set of rules. These rules are based on three introduced measures: representativeness, seclusion, and potential belonging.

In Agarwal et al. (2012), the authors address the problem of discovering events in a microblog stream. To this extent, they mapped the problem of finding events to that of finding clusters in a graph. The authors describe aMQCs: biconnected clusters, satisfying *short-cycle property* that allow them to find and maintain the clusters locally without affecting their quality. Network dynamics are handled through a rule-based approach for addition/deletion of nodes/edges.

In Duan et al. (2012), social networks' dynamics are modeled as a *change stream*. Based on this model, a local DFS forest updating algorithm is proposed for incremental 2-clique clustering, and it is generalized to incremental k-clique clustering. The incremental strategies are designed to guarantee the accuracy of the clustering result with respect to any kind of changes (i.e., edge addition/deletion). Moreover, the authors show how the local DFS forest updating algorithm produces not only the updated connected components of a graph but also the updated DFS forest, which can be applied to other issues, such as finding a simple loop through a node.

In Görke et al. (2012), the authors show that the structure of *minimum-s-t-cuts* in a graph allow for an efficient dynamic update of *minimum-cut trees*. The authors proposed an algorithm that efficiently updates specific parts of such a tree and dynamically maintains a graph clustering based on minimum-cut trees under arbitrary atomic changes. The main feature of the graph clustering computed by this method is that it is guaranteed to yield a certain expansion—a bottleneck measure—within and between clusters, tunable by an input parameter α. The algorithm ensures that its community updates handle temporal smoothness—for instance, changes to the clusterings are kept at a minimum, whenever possible.

The CUT algorithm, introduced in Ma and Huang (2013), tracks community seeds to update community structures instead of recalculating them from scratch as time goes by. The process is decomposed into two steps:

(1) *First snapshot*: Identify community seeds (collection of 3-cliques).
(2) *Successive snapshots*: (i) Track community seeds and update them if changes occur and (ii) expand community seeds to complete communities.

To easily track and update community seeds, CUT builds up a clique adjacent bipartite graph (CAB). Custom policies are then introduced to handle node/edge join/removal and seed community expansion.

In Xie et al. (2013a), the LabelRankT algorithm is introduced. It is an extension of LabelRank (Xie and Szymanski 2013), an online distributed algorithm for the detection of communities in large-scale dynamic networks through stabilized label propagation. LabelRankT takes advantage of the partitions obtained in previous snapshots for inferring the dynamics in the current one.

The method described in Lee et al. (2014) aims at monitoring the evolution of a graph applying a fading time window. The proposed approach maintains a skeletal graph that summarizes the information in the dynamic network. Communities are defined as the connected components of this skeletal graph. Communities on the original graphs are built by expanding the ones found on the skeletal graph.

In Zakrzewska and Bader (2015), the authors propose an algorithm for dynamic greedy seed set expansion, which incrementally updates the traced community as the underlying graph changes. The algorithm incrementally updates a local community starting from an initial static network partition (performed through a classic set seed expansion method). For each update, the algorithm modifies the sequence of community members to ensure that corresponding fitness scores are increasing.

In Rossetti et al. (2017), the authors introduce TILES,[5] an online algorithm that dynamically tracks communities in an edge stream graph following local topology perturbations. Exploiting local patterns and constrained label propagation, the proposed algorithm reduces the computation overhead. Communities are defined as two-level entities:

[5]The TILES code is available at https://goo.gl/zFRfCU.

— *Core node*: A core node is defined as a node involved in at least a triangle with other core nodes of the same community.
— *Community periphery*: The set of all nodes at one hop from the community core.

Event detection is also performed at runtime (birth, split, merge, death, expansion, contraction).

B.4 Informed CD by Network Smoothing

In Kim and Han (2009), the authors propose a particle-and-density–based method for efficiently discovering a variable number of communities in dynamic networks. The method models a dynamic network as a collection of particles called *nanocommunities*, and model communities as a densely connected subset of such particles. Each particle, to ensure the identification of dense substructures, contains a small amount of information about the evolution of neighborhoods and/or communities and their combination, both locally and across adjacent network snapshots (which are connected as t-partite graphs). To allow flexible and efficient temporal smoothing, a cost embedding technique—independent on both the similarity measure and the clustering algorithm used—is proposed. In this work, a mapping method based on information theory is also proposed.

The method proposed in Guo et al. (2014) finds communities in each dynamic weighted network snapshot using modularity optimization. For each new snapshot, an input matrix, which represents a trade-off between the adjacency matrix of the current step and the adjacency matrix of the previous step, is computed. Then a custom modularity optimization community detection algorithm is applied.

In Xu et al. (2013a, 2013b), the cumulative stable contact (CSC) measure is proposed to analyze the relationship among nodes: upon such measure, the authors build an algorithm that tracks and updates stable communities in mobile social networks. The key definition provided in this work concerns the following:

— *Cumulative stable contact*: There is a CSC between two nodes if and only if their history contact duration is higher than a threshold.
— *Community core set*: The community core at time t is a partition of the given network built upon the useful links rather than on all interactions.

The network dynamic process is divided into snapshots. Nodes and their connections can be added or removed at each snapshot, and historical contacts are considered to detect and update the community core set. Community cores are tracked incrementally, allowing recognition of evolving community structures.

C CROSS-TIME COMMUNITIES DISCOVERY

Here, we collected and classified all works introducing approaches falling in the Cross-Time class. For details on the class definition and peculiarities, refer to Section 4.3.

C.1 Fixed Memberships, Fixed Properties

GraphScope (Sun et al. 2007) is an algorithm designed to find communities in dynamic bipartite graphs. Based on the minimum description length (MDL) principle, it aims to optimize graph storage while performing community analysis. Starting from an SN, it reorganizes snapshots into segments. Given a new incoming snapshot, it is combined with the current segment if there is a storage benefit; otherwise, the current segment is closed, and a new one is started with the new snapshot. The detection of communities sources and destination for each segment is done using MDL.

In Duan et al. (2009), the authors introduce a method called Stream-Group, which is designed to identify communities on dynamic directed weighted graphs (DDWGs). Stream-Group relies on a two-step approach to discover the community structure in each time slice:

(1) The first step constructs compact communities according to each node's single compactness—a measure that indicates the degree a node belongs to a community in terms of the graph's relevance matrix.
(2) In the second step, compact communities are merged along the direction of maximum increase of the modularity.

A measure of the similarity between partitions is then used to determine whether a change point appears along the time axis and an incremental algorithm is designed to update the partition of a graph segment when adding a new arriving graph into the graph segment. In detail, when a new time slice arrives,

(1) the community structure of the arriving graph is computed;
(2) the similarity between the partition of the new arriving graph and that of the previous graph segment is evaluated;
(3) change point detection is applied: if the time slice t is not a change point, then
(4) the partition of the graph is updated; otherwise, a new one is created.

In Aynaud and Guillaume (2011), the authors propose a definition of average modularity Q_{avg} over a set of snapshots (called a *time window*), defined as a weighted average of modularity for each snapshot, with weights defined a priori—for instance, to reflect heterogeneous durations. Optimizing average modularity yields a constant partition relevant over all considered snapshots. This approach does not contemplate community operations.

The authors propose two methods to optimize this modularity:

—*Sum method*: Given an evolving graph $G = \{G_1, G_2, \ldots, G_n\}$ and a time window $T \subseteq \{1, \ldots, n\}$, a cumulative weighted graph is built, called the *sum graph*, which is the union of all snapshots in T. Each edge of the sum graph is weighted by the total time during which this edge exists in T. Since the sum graph is a static weighted graph, the authors apply the Louvain method (Blondel et al. 2008) on it. This method is not strictly equivalent to optimizing Q_{avg} but allows a fast approximation.
—*Average method*: Two elements of the Louvain method (Blondel et al. 2008) are changed to optimize the average modularity during a time window T: (i) the computation of the quality gain in the first phase and (ii) how to build the network between communities in the second one:
  (1) Average modularity gain is defined as the average of the static gains for each snapshot of T.
  (2) Given a partition of the node set, the same transformation of Louvain is applied to every snapshot of T independently (with different weights for each snapshot) to obtain a new evolving network between the communities of the partition.

The authors finally propose a method using sliding windows to segment the evolution of the network in stable periods, potentially hierarchically organized.

C.2 Fixed Memberships, Evolving Properties

In Gauvin et al. (2014), the authors propose to use a nonnegative tensor factorization approach. First, all adjacency matrices, each corresponding to a network snapshot, are stacked together in a three-way tensor (a three-dimensional matrix with a dimension corresponding to time). A custom

nonnegative factorization method is applied, with the number of desired communities as input, that yields two matrices: one corresponds to the membership weight of nodes to components and the other to the activity level of components for each time corresponding to a snapshot. It can be noted that each node can have a nonzero membership to several (often all) communities. A postprocess step is introduced to discard memberships below a given threshold and thus simplify the results.

In Matias and Miele (2016), the authors propose to use a Poisson process stochastic block model (PPSBM) to find clusters in temporal networks. As in a static SBM, nodes belong to groups, and each pair of groups is characterized by unique connection characteristics. Unlike static SBM, such a connection characteristic is not defined by a single value but by a function of time f (intensity of conditional inhomogeneous Poisson process) representing the probability of observing interactions at each point in time. The authors propose an adapted EM to find both the belonging of nodes and f that better fit the observed network. It is to be noted that nodes do not change their affiliation using this approach; instead, it is the clusters' properties that evolve along time.

C.3 Evolving Memberships, Fixed Properties

In Yang et al. (2009, 2011) the authors propose using a dynamic SBM. As in a typical static SBM, nodes belong to clusters, and an interaction probability β_{ql} is assigned to each pair of clusters (q, l). The dynamic aspect is handled by a transition matrix that determines the probability for nodes belonging to a cluster to move to each other cluster at each step. The optimal parameters of this model are searched for using a custom EM algorithm. The β_{ql} are constant for the same community for all timesteps.

Two versions of the framework are discussed:

— *online learning*, which updates the probabilistic model iteratively (in this case, the method is a Temporal Trade-Off CD), and
— *offline learning*, which learns the probabilistic model with network data obtained at all timesteps (in this case, the method is a Cross-Time CD).

Variations with a similar rationale have been proposed by different authors, most notably in Ishiguro et al. (2010), Herlau et al. (2013), and Xu and Hero (2014).

In Matias et al. (2015), the authors propose using a dynamic SBM similar to Yang et al. (2009). As in a typical static SBM, nodes belong to clusters, and an interaction probability β_{ql} is assigned to each pair of clusters (q, l). The dynamic aspect is handled as an aperiodic stationary Markov chain, defined as a transition matrix π, characterizing the probability of a node belonging to a given cluster a at time t to belong to each one of the other clusters at time $t + 1$. The optimal parameters of this model are searched for using a custom EM algorithm. The authors note that it is not possible to allow the variation of β and node memberships parameters at each step simultaneously and therefore impose a single value for each intragroup connection parameter β_{qq} along the whole evolution, thus searching for clusters with stable intragroup interaction probabilities. Finally, the authors introduce a method for automatically finding the best number of clusters using integrated classification likelihood (ICL) and an initialization procedure allowing to converge to a better local maximum using an adapted k-means algorithm.

In Ghasemian et al. (2016), the authors also use a dynamic SMB similar to Yang et al. (2009) and propose scalable algorithms to optimize the parameters of the model based on belief propagation and spectral clustering. They also study the detectability threshold of the community structure as a function of the rate of change and the strength of communities.

C.4 Evolving Memberships, Evolving Properties

In Jdidia et al. (2007), the authors propose adding edges between nodes in different snapshots (i.e., edges linking nodes at t and $t + 1$). Two types of these edges can be added:

— *identity edges* between the same node, if present at t and $t + 1$, and
— *transversal edges* between different nodes in different snapshots.

To do so, the following relation must hold: there is an edge between $u \in t$ and $v \in t + 1$ if $\exists w$ such that $(u, w) \in t$ and $(v, w) \in t + 1$. A static CD algorithm, Walktrap, is consequently applied to this transversal network to obtain dynamic communities.

The method introduced in Mucha et al. (2010) was designed for any multisliced network, including evolving networks. The main contribution lies in a multislice generalization of modularity that exploits Laplacian dynamics. This method is equivalent to adding links between same nodes in different slices and then running a community detection on the resulting graph. In the article, edges are added only between same nodes in adjacent slices. The weight of these edges is a parameter of the method. The authors underline that the resolution of the modularity can vary from slice to slice. In Bassett et al. (2013), variations of the same method using different null models are explored.

In Viard et al. (2016), the notion of clique is generalized to link streams. A Δ-clique is a set of nodes and a time interval such that all nodes in this set are pairwise connected at least once during any subinterval of duration Δ of the interval. The work presents an algorithm able to compute all maximal (in terms of nodes or time interval) Δ-cliques in a given link stream, for a given Δ. A solution able to reduce the computational complexity is proposed in Himmel et al. (2016).